

SPRING CORE CHEAT SHEET

Container, Dependency and IOC



Spring 5.x - Date : August 2018

WHAT IS SPRING ?

- **Open source** (Apache 2 licence, sources on [github](#))
- **Light**
 - Doesn't force to use an application server
 - Not invasive
- **Container**
 - Application objects doesn't have to look for their dependencies
 - Handles objects life cycle
- **Framework**
 - Ease integration and communication with third-party libraries

MINIMAL DEPENDENCIES

```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
</dependency>
```

APPLICATION CONFIGURATION

- Java
- Annotations
- XML (still available but heavy)

```
@ComponentScan("fr.sii.cheatsheet.spring")
public class MyApp {

    public static void main(String[] args) {
        ApplicationContext app = new
        AnnotationConfigApplicationContext(MyApp.class);

        DummyService helloWorld =
        app.getBean(DummyService.class);
        helloWorld.getMessage();
    }
}
```

Prefer use of a configuration class

```
@Configuration
public class MyAppConfig {
    // @Bean, ...
}
```

DEPENDENCY INJECTION

Define a bean : **@Bean**

```
@Configuration
public class MyAppConfig {
    @Bean
    public DummyService dummyService(){
        return new DummyServiceImpl();
    }
}
```

Define a bean : **@Component**

@Component specializations :

- **@Service** : Heart of an app
- **@Repository** : Handles data persistence
- **@Controller** : Handles requests and responses

```
@Component
public class DummyServiceImpl implements
DummyService {
}
```

Dependency injection : **@Autowired**

- Via a class field
- Via a setter
- Via the constructor (*prefer for easy test*)

```
@Component
public class FooServiceImpl implements FooService
{
```

```
    @Autowired
    private DummyService service;
```

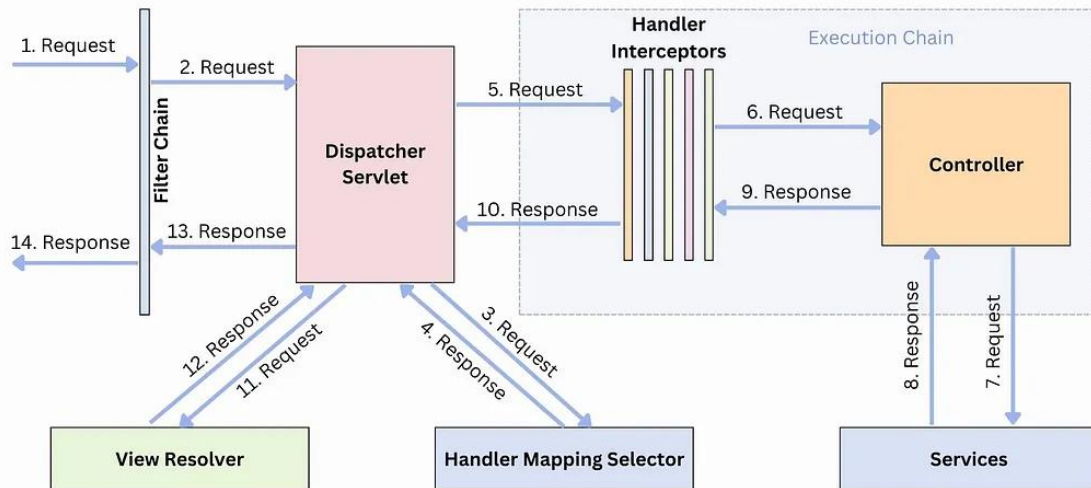
```
    @Autowired
    public FooServiceImpl(DummyService
dummyService) {
        this.service = dummyService;
    }
```

```
    @Autowired
    public DummyService
setDummyService(DummyService dummyService) {
        this.service = dummyService;
    }
}
```

Spring MVC Basic CheatSheet

- **Model** - application data, typically POJOs
- **View** - displays Model data, often generating frontend
- **Controller** handles requests, creates the corresponding Model, and passes it for display in the View.

In REST applications, **RestController** returns domain objects or collections directly. Usually response body is a JSON object



Annotations

@Controller	Marks a class as a Spring MVC controller, capable of handling HTTP requests.
@RequestMapping	Maps a method or class to a specific URL or pattern. Specifies which HTTP methods the method can handle.
@GetMapping @PostMapping @PutMapping @DeleteMapping	Shortcuts for mapping HTTP GET, POST, PUT, and DELETE requests, respectively, without using the @RequestMapping annotation.
@RequestParam	Binds a request parameter to a method parameter. Used to access query parameters or form data from HTTP requests.
@PathVariable	Extracts values from URL path segments and binds them to method parameters.
@ModelAttribute	Binds a method parameter or method return value to a named model attribute. Used for populating data in the model before rendering a view.
@ResponseBody	Indicates that the return value of a method should be used as the HTTP response body, often used for RESTful services returning data in JSON or XML.
@RequestBody	Binds the HTTP request body to a method parameter, typically used for receiving data in JSON or XML format.
@RestController	Combines @Controller and @ResponseBody to simplify the creation of RESTful web services.
@ExceptionHandler	Defines methods to handle exceptions thrown by the controller methods.
@ResponseStatus	Specifies the HTTP status code to be returned when a specific exception is thrown.
@SessionAttributes	Specifies which model attributes should be stored in the session between requests.
@RequestHeader	Binds a request header value to a method parameter.
@CrossOrigin	Enables cross-origin resource sharing (CORS) for specific controller methods.
@ResponseStatus	Specifies the HTTP response status code and reason phrase to be returned by a controller method.